

SkillAudit: Traceable Validation of Simplified Paper-Derived Agent Skill Specifications

Anonymous submission

Abstract

Large Language Model (LLM) agents can convert method descriptions in research papers into executable agent skill specifications, but simplifying these specifications may remove information required for method reproduction. Existing approaches to this task mainly focus on extracting method content and generating executable specifications, whereas methods for compressing prompts or skill specifications primarily optimize text length, execution cost, or downstream performance. However, these two lines of work rarely jointly record which source passages were removed, whether the remaining steps preserve all required dependencies, and which experimental results support the decision to accept or reject a simplified specification. In this paper, we present SkillAudit, a traceable post-compression validation protocol for candidate specifications produced by upstream simplification methods. It links procedural steps to their source passages and prerequisites, compares performance across three conditions (a no-specification baseline, the full specification, and a simplified candidate specification) on prespecified tasks, applies prespecified acceptance criteria, and produces an auditable record containing source provenance, the experimental configuration, test results, and the decision rationale. We construct code-based method-reproduction tasks from algorithmic mechanisms selected from different papers to evaluate SkillAudit. The results show that SkillAudit distinguishes failures due to an insufficient full specification, failures due to missing critical information in a simplified candidate specification, and invalid experimental runs.

Introduction

Large Language Model (LLM) agents can use natural-language instructions, external tools, and execution environments to complete complex multi-step tasks. As agent capabilities advance, method descriptions in research papers are increasingly treated as reusable sources of procedural knowledge. Existing studies use agents to recreate code and experiments described in papers (Starace et al. 2025; Zhao et al. 2026), while paper-to-agent systems further transform papers and associated software into interactive agent artifacts (Miao et al. 2025). Through such upstream conversion

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

processes, paper methods can be organized into executable agent skill specifications.

Existing research follows two related directions. The first extracts method content from papers or other knowledge sources and produces executable agent artifacts or reusable skills (Miao et al. 2025; Shen et al. 2026; Pan et al. 2026). The second reduces context length and execution cost through prompt compression or agent-skill compression (Jiang et al. 2023, 2024; Pan et al. 2024; Gao et al. 2026). For example, LLMLingua removes tokens estimated to carry limited information, LongLLMLingua introduces question-aware compression for long contexts, and LLMLingua-2 formulates compression as token-level classification. However, removing tokens or sentences can disrupt semantic coherence and discard necessary information at high compression ratios (Pan et al. 2024; Liskavets et al. 2025).

A shorter skill specification does not necessarily retain all information required for method reproduction. Existing methods commonly evaluate simplification using compression ratio, execution cost, or downstream task performance, but rarely jointly record which source passages were removed, whether the remaining steps preserve their prerequisites, and which experimental results justify accepting or rejecting a candidate. Consequently, success on a small set of examples does not distinguish an insufficient full specification, a simplified candidate missing critical information, and an invalid experimental run. We therefore study candidate-level post-compression validation: given a full skill specification, a simplified candidate specification, and an executable evaluation package, determine whether the candidate can substitute for the full specification for a specified task under fixed experimental conditions.

We present SkillAudit, a traceable post-compression validation protocol. SkillAudit takes a source-to-step mapping confirmed by the evaluation-package builder, binds procedural steps to source passages and prerequisites, and checks whether a simplified candidate preserves the dependency closure required for task execution. Before held-out evaluation, the protocol freezes the candidate, task, workspace, model, tools, run budget, tests, scorer, and prespecified acceptance criteria. It then executes three conditions: a no-specification baseline, the full specification, and the simplified candidate specification. Finally, SkillAudit combines held-out test results with integrity checks to distinguish task failures from

invalid runs, produces an Accept, Reject, or Invalid decision, and emits an auditable validation record containing source provenance, experimental configuration, results, and the decision rationale.

.....palceholder Inspired by these insights, we introduce AutoTool, a novel graph-based method for automatic tool selection in LLM agents. AutoTool constructs a graph representation capturing the observed inertia in tool selection from historical workflows, where nodes correspond to tools and edges encode observed sequential dependencies. Additionally, AutoTool intelligently integrates parameter-level information into this graph, thereby enabling automated parameter filling. By traversing this structured representation, AutoTool selects tools and parameters significantly faster than traditional inference-based approaches. Experimental evaluations demonstrate that AutoTool substantially reduces token consumption and LLM call counts while maintaining comparable agent task progress rates to LLM-based methods. Our contributions are summarized as follows:

Our contributions are threefold:

- We formulate candidate-level post-compression validation and distinguish failures caused by an insufficient full specification, missing information in a simplified candidate, and invalid experimental runs.
- We introduce SkillAudit, which integrates source mapping, dependency-closure checking, controlled three-condition execution, a frozen evaluation boundary, and auditable decision records into a unified validation protocol.
- We construct code-based method-reproduction tasks to evaluate SkillAudit across paper-derived mechanisms, tasks, and execution conditions.

Related Work

From Papers to Executable Research Artifacts

Recent systems increasingly treat papers as inputs to executable research workflows. PaperBench evaluates whether agents can reproduce machine-learning research from paper descriptions, while AutoReproduce, RExBench, and REPRO-Bench study experiment reproduction, research-extension implementation, and reproducibility assessment under different forms of paper and repository evidence (Starace et al. 2025; Zhao et al. 2026; Edwards et al. 2026; Hu et al. 2025). A complementary line of work constructs reusable artifacts rather than only scoring reproduction. Paper2Agent turns papers and code into interactive agents, executable knowledge graphs retain research operations and dependencies, and skill-compilation systems convert heterogeneous scientific resources into reusable agent instructions (Miao et al. 2025; Luo et al. 2026; Shen et al. 2026; Pan et al. 2026). The AAI 2025 KTCE system similarly creates reusable and diverse Python toolsets from domain knowledge and improves them through evolutionary search (Ma et al. 2025).

These systems primarily ask whether a complete artifact can be constructed or used. Their outputs are implementa-

tions, tools, agents, execution traces, or aggregate reproduction scores. SkillAudit addresses a later and narrower claim: after an upstream system has produced a full specification and a simplified candidate, can the fixed candidate replace the full specification for a declared task? This change of unit matters because a plausible paper-to-agent artifact may still be incomplete, while a shorter candidate may fail for a reason that cannot be attributed to compression unless the full specification succeeds under the same boundary.

Compression, Reduction, and Reusable Skills

Prompt compression reduces model-facing context while attempting to preserve downstream behavior. LLMingua, LongLLMingua, and LLMingua-2 use token importance, question-aware compression, and data-distilled token classification, respectively (Jiang et al. 2023, 2024; Pan et al. 2024). More recent methods retain sentence structure or optimize compression around downstream errors and long-context perception (Liskavets et al. 2025; Larionov and Eger 2025; Tang et al. 2025). In particular, the AAI 2025 context-aware method defines compression as selecting a shorter context whose task performance matches the original, then learns sentence relevance conditioned on the query (Liskavets et al. 2025). Agent-oriented systems extend this objective from prompts to reusable skills, optimizing skill tokens, execution cost, retrieval, or trace reuse (Gao et al. 2026; Xu and Wu 2026; Yuan et al. 2026; Liu et al. 2026; Meng, Wang, and Fang 2026).

Program reduction offers a useful structural precedent. Delta debugging, C-Reduce, cause reduction, and syntax-guided reduction repeatedly remove program parts while consulting an external predicate (Zeller and Hildebrandt 2002; Regehr et al. 2012; Groce et al. 2016; Sun et al. 2018). Skill simplification differs because natural-language procedural units have source provenance, latent dependencies, and stochastic executors, but it inherits the same central lesson: shortness alone is not evidence that the target behavior was preserved. Existing compression work usually reports aggregate quality, length, latency, or cost. It rarely combines exact deleted source spans, dependency-closure checks, a matched full-specification control, held-out execution, and a candidate-specific acceptance record.

Agent Evaluation and Selective Decisions

Tool-learning studies measure whether models can invoke tools while retaining general capabilities. Toolformer learns when and how to call APIs; Confucius and CITI improve tool learning through curriculum or component-aware adaptation; AutoTool uses historical tool inertia to bypass predictable LLM calls (Schick et al. 2023; Gao et al. 2024; Hao et al. 2025; Jia and Li 2026). AgentBoard evaluates multi-turn progress, MCP-AgentBench tests protocol-mediated real-world tool use, and Beyond Accuracy analyzes capability boundaries beyond a single end score (Ma et al. 2024; Guo et al. 2026; Wang et al. 2026). These benchmarks provide scores, traces, and capability profiles for complete agents. They do not decide whether one simplified paper-derived specification is an acceptable substitute for its full counterpart.

SkillAudit is also related to selective prediction and guarded evaluation. SelectiveNet learns to abstain on uncertain predictions, conformal risk control bounds selected risks under stated assumptions, and reusable-holdout work studies repeated access to hidden evidence (Geifman and El-Yaniv 2019; Angelopoulos et al. 2022; Dwork et al. 2015; Nakkiran and Błasiok 2018). We do not claim conformal coverage or population-level error control. Instead, SkillAudit makes a finite engineering decision under prespecified thresholds and emits INVALID when the evidence boundary is broken. Its contribution is the conjunction of three properties that prior lines usually separate: source-addressed structural provenance, matched behavioral substitution evidence, and a traceable ACCEPT/REJECT/INVALID record for one fixed candidate.

Problem Formulation and SkillAudit

Problem Formulation

Let a paper P describe a bounded procedure to be reused by an LLM agent. An upstream extraction system produces a *full specification* F , and an upstream reducer produces a shorter *candidate specification* S . We do not assume that F is complete or that S is faithful. Given a task t and an evaluation boundary Ω , our question is whether S can substitute for F without making the task solvable from the task statement alone. We therefore compare three artifact conditions: B , a no-specification baseline; F , the full specification; and S , the simplified candidate.

The paper procedure is represented by source-addressed atoms $A = \{a_1, \dots, a_K\}$ and a directed acyclic dependency graph $G = (A, E)$. Each atom is

$$a_i = (\text{id}_i, \ell_i, u_i, r_i, D_i), \quad (1)$$

where ℓ_i is an exact locator in the identified paper, u_i is an executable instruction, r_i records its procedural role, and D_i contains the identifiers of its direct prerequisites. A candidate atom set $A_S \subsetneq A$ is admissible for evaluation only if it is nonempty, acyclic, and dependency closed: $D_i \subseteq \{\text{id}_j : a_j \in A_S\}$ for every $a_i \in A_S$. A deterministic renderer R then produces $F = R(A)$ and $S = R(A_S)$. This representation proves which source content was retained or removed; it does not prove that either artifact is behaviorally sufficient.

For each registered block $r \in \{1, \dots, n\}$ and condition $C \in \{B, F, S\}$, an agent receives the same task, starter workspace, tools, public test, action budget, and private scorer. Only the assigned artifact changes. The resulting terminal row contains a validity bit $v_{C,r}$, a hard-contract bit $h_{C,r}$, a patch-present bit $p_{C,r}$, and a private score $q_{C,r} \in [0, 1]$. With a prespecified score threshold τ , operational success is

$$z_{C,r} = \mathbb{I}[v_{C,r} = 1 \wedge h_{C,r} = 1 \wedge p_{C,r} = 1 \wedge q_{C,r} \geq \tau]. \quad (2)$$

The block count $k_C = \sum_{r=1}^n z_{C,r}$ is descriptive evidence for this finite schedule, not an estimate of a universal success probability.

Algorithm 1: SkillAudit for one fixed candidate

Require: Paper P , atoms (A, G) , candidate A_S , task t , boundary Ω

- 1: Verify paper identity, source locators, DAG validity, and closure of A_S
- 2: Render and freeze $F = R(A)$, $S = R(A_S)$, and baseline B
- 3: **for** each registered block r and condition $C \in \{B, F, S\}$ **do**
- 4: Execute a fresh agent session and retain the complete terminal output
- 5: Validate digests, submission, patch, scorer, and hard task contract
- 6: **end for**
- 7: **if** the registered schedule or an integrity check is incomplete **then**
- 8: $\delta \leftarrow \text{INVALID}$
- 9: **else**
- 10: Compute k_B, k_F, k_S and apply the frozen acceptance rule
- 11: $\delta \leftarrow \text{ACCEPT}$ if all fields pass; otherwise REJECT
- 12: **end if**
- 13: Emit source links, Ω , terminal outputs, field-level checks, and δ

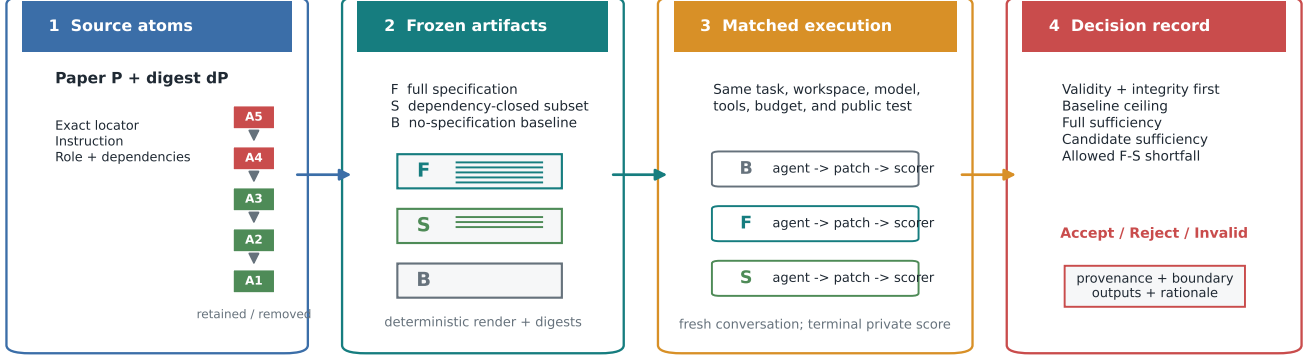
SkillAudit Overview

Figure 1 shows the complete information flow. SkillAudit is a post-compression validator rather than a reducer: it accepts candidates generated by a heuristic, learned, or interactive upstream method, but never edits a candidate after held-out evaluation begins. The protocol first verifies the source map and dependency closure, then freezes the artifacts and experimental boundary, executes matched $B/F/S$ blocks, validates every terminal row, applies a prespecified decision rule, and emits an evidence record. Algorithm 1 states the same procedure operationally.

Source-Addressed Candidate Construction

The atom registry is constructed from quoted paper passages. Exact document digests and page, line, or byte spans make the mapping inspectable, while r_i distinguishes operations, parameters, guards, ordering constraints, and output contracts. Dependencies are recorded separately from prose order because a prerequisite may be stated in another paragraph, caption, or equation. The closure auditor rejects unknown identifiers, graph cycles, a non-strict subset, or a retained atom whose prerequisite is absent. Rendering uses a fixed topological order with atom-identifier tie breaking, so the same registry produces byte-identical artifacts.

The candidate boundary remains explicit. For example, if a_4 implements a solver and depends on preprocessing atoms a_1 – a_3 , retaining a_4 while removing a_2 is a structural failure before any model call. Removing a_4 while retaining a_1 – a_3 can be structurally valid but behaviorally insufficient. This separation prevents a dependency check from being overinterpreted as a reproduction guarantee. It also permits failure analysis to point back from a rejected candidate to the



Only the assigned artifact changes across B/F/S; every decision remains local to the frozen evaluation boundary.

Figure 1: End-to-end SkillAudit protocol. A paper is decomposed into source-addressed atoms and explicit dependencies; green atoms are retained by the candidate and red atoms are removed. The full specification F , simplified candidate S , and no-specification baseline B are frozen and run under matched task and execution conditions. Validity and integrity are checked before the finite-schedule gate produces ACCEPT, REJECT, or INVALID. The evidence record binds provenance, the frozen boundary, terminal outputs, and the decision rationale.

exact source units that were omitted.

Frozen Evaluation Boundary

The evaluation boundary binds every factor that may change the interpretation of a comparison:

$$\Omega = (P, A, G, F, S, t, W, M, T, U, Q, \sigma, \tau, \gamma), \quad (3)$$

where W is the starter workspace, M the model and decoding configuration, T the tool interface, U the action budget, Q the public and private tests plus scorer, σ the blocked condition schedule, τ the operational-success threshold, and γ the acceptance thresholds. Digests bind the concrete files associated with these symbols. The block order is balanced when the schedule permits, and every condition starts from an isolated workspace and fresh conversation.

Public tests are visible development feedback: the agent may run them before submission. The private scorer is terminal and one-way. Once the agent submits a patch, the patch is frozen, the private cases are executed once, and no private result is returned to the agent. This boundary limits direct adaptation to hidden cases while preserving a realistic development loop. Case-generation seeds, registry assignments, and condition order are recorded. When a provider does not expose an enforceable sampling seed, the record explicitly distinguishes deterministic orchestration from nondeterministic model execution.

Validity Before Performance

SkillAudit separates execution validity from task failure. A valid scored failure includes a submitted patch that violates the hard output contract or falls below τ ; it contributes $z_{C,r} = 0$ to the registered denominator. A row is invalid when the registered request was not executed as specified, an expected output or digest is missing, the scorer did not complete, or

the provider/model was unavailable after the retry policy. Malformed or empty submissions are recorded as their own terminal category and are interpreted according to the frozen row-validity rule. This taxonomy prevents network, wrapper, serialization, and scoring defects from being silently counted as evidence that a skill is insufficient.

Let $V = 1$ only when all required rows and schedule-level integrity checks are complete. With a baseline ceiling $b_{\max}$, minimum full and candidate counts $f_{\min}$ and $s_{\min}$, and maximum allowed shortfall ϵ , the decision is

$$\delta_{t,\Omega}(S) = \begin{cases} \text{INVALID}, & V = 0, \\ \text{ACCEPT}, & V = 1, k_B \leq b_{\max}, k_F \geq f_{\min}, \\ & k_S \geq s_{\min}, k_F - k_S \leq \epsilon, \\ \text{REJECT}, & \text{otherwise.} \end{cases} \quad (4)$$

The baseline ceiling checks that the task is not already easy without a specification. Full sufficiency checks that the reference artifact and evaluation package can support the task. Candidate sufficiency and the $F - S$ shortfall test the substitution claim. A rejection is local: it withholds permission to substitute under (t, Ω) and does not declare the candidate semantically wrong for every task or model.

Auditable Evidence Record

The output is not only a label. SkillAudit emits

$$\mathcal{E} = (d_P, d_A, d_F, d_S, d_\Omega, \mathcal{O}, \mathcal{C}, \delta, \rho), \quad (5)$$

where the d terms identify the paper, atom registry, artifacts, and boundary; $\mathcal{O}$ is the immutable set of requests, trajectories, patches, scores, and transport attempts; $\mathcal{C}$ contains field-level validity and acceptance checks; and ρ is a human-readable rationale. Consequently, an accepted candidate can be distributed with the exact task and evidence that support its

use, while a rejected or invalid candidate carries a diagnosis rather than an unqualified failure label.

Experiments

Research Questions

We organize the evaluation by the claim each experiment tests rather than by the order in which artifacts were produced.

- **RQ1: Diagnostic validity.** Can matched $B/F/S$ execution distinguish an insufficient full specification from information lost only by the simplified candidate?
- **RQ2: Decision sanity.** Do strict-subset, negative, and byte-identical controls exercise the intended acceptance, rejection, and invariance states?
- **RQ3: Scale and failure localization.** What happens when the protocol is expanded across papers, tasks, domains, registries, reducers, and models, and can it separate task failures from interface failures?

Experimental Setup

Focused validation suite. The focused suite contains two code-based method-reproduction tasks. SNAP-MFSE isolates the matrix-free spectral-embedding mechanism in SnapATAC2, including normalization, degree construction, operator application, and the eigensolver guard (Zhang et al. 2024). Toolformer filtering implements the rule for retaining candidate API calls, including future-token weights, weighted losses, the stronger counterfactual, inclusive thresholding, and ordered application (Schick et al. 2023). Each strict-subset family uses 18 matched blocks; every private block contains 64 deterministic cases and hard interface checks. We set $\tau = 0.95$, so at least 61/64 cases must pass, and use $(b_{\max}, f_{\min}, s_{\min}, \epsilon) = (2, 16, 16, 2)$. Identity instrumentation uses six paired blocks and a separately frozen 5/6 agreement rule.

The evaluated candidates are deliberately interpretable. The SNAP candidate retains the first three of five atoms and removes the matrix-free operator and solver/guard units. Toolformer prefix 01 retains only the first of five atoms and acts as a real negative. Prefix 04 retains four of five atoms and removes the all-candidate/order unit. A planted-positive control removes a redundant sixth atom, while identity instrumentation presents byte-identical F and S . The focused suite uses DeepSeek V3.2 with temperature zero; the provider does not expose an enforceable sampling seed.

Cross-paper stress test. The broader registered schedule contains 12 papers and two tasks per paper across NLP, software engineering, data analysis, and agent/tool use. Its mechanisms include LLMingua, LLMingua-2, context-aware sentence compression, delta debugging, C-Reduce, Perses, HDBSCAN, Leiden, SnapATAC2, ReAct, Reflexion, and Toolformer (Jiang et al. 2023; Pan et al. 2024; Liskavets et al. 2025; Zeller and Hildebrandt 2002; Regehr et al. 2012; Sun et al. 2018; Campello et al. 2015; Traag, Waltman, and van Eck 2019; Zhang et al. 2024; Yao et al. 2023; Shinn et al. 2023; Schick et al. 2023). The primary candidate retains a mean token ratio of 0.579 (range 0.561–0.598). Two disjoint private registries, A and B, receive 648 rows each.

Registered family	Rows
Primary $B/F/S$ comparisons	432
Controls	144
Structural retention ladder	144
Alternate reducers	96
Closed-model sentinels	384
Remote execution anchor	96
Total	1,296

Table 1: Composition of the cross-paper registered stress test. The 432 primary rows are 24 tasks $\times$ 6 blocks $\times$ 3 artifact conditions. The remaining rows test the measurement boundary rather than increase the primary task denominator.

Table 1 reports the full 1,296-row schedule. The primary family contains the 24 task-level $B/F/S$ comparisons. Additional families test controls, structural retention levels, alternate reducers, closed-model sentinels, and a remote execution anchor. DeepSeek V3.2 contributes 816 rows; GPT-5.5, GPT-5.6 Sol, GPT-5.6 Terra, GPT-5.6 Luna, and Claude Opus 4.7 contribute 96 rows each. The schedule records deterministic seeds for case generation, registry assignment, and ordering. These seeds do not control opaque provider-side sampling, so we make no claim of backend determinism.

Metrics and evidence units. The unit of model execution is a fresh conversation; the unit of private evaluation is a terminal patch scored over one 64-case block. Cases within a block characterize the patch and are not treated as independent agent runs. We report private score q , operational success z , condition success counts k_C , the finite-schedule decision in Equation 4, terminal failure categories, candidate token ratio, and evidence cost. All reported counts are recomputed from terminal rows, not from a provider’s textual self-assessment.

RQ1: Distinguishing Full and Candidate Insufficiency

Table 2 and Figure 2 report the complete focused schedules. SNAP produces $B/F/S$ success counts of 0/15/5. Because F misses the 16/18 full-sufficiency requirement, the protocol rejects the substitution claim without attributing the failure solely to simplification. This is the intended “full insufficient” state: improving S alone would not repair the reference artifact or evaluation package.

Toolformer prefix 01 produces 0/17/8. Here F clears the full-sufficiency gate, but the one-atom candidate misses candidate sufficiency by eight blocks and trails F by nine. SkillAudit therefore isolates candidate-specific insufficiency under the same task and scorer. Prefix 04 produces 0/18/17 and satisfies every acceptance field. It is a non-planted strict subset accepted for this task and boundary. Because prefix 04 was selected after inspecting the earlier focused result and then evaluated on a separately frozen, nonoverlapping registry, its conclusion is explicitly local rather than a prospectively estimated admission rate.

Family	Blocks	B success	F success (mean q)	S success (mean q)	$F - S$ gap	Status
SNAP 3/5 candidate	18	0	15 (0.877)	5 (0.321)	10	Reject: full insufficient
Toolformer 1/5 candidate	18	0	17 (0.944)	8 (0.481)	9	Reject: candidate insufficient
Toolformer 4/5 candidate	18	0	18 (1.000)	17 (0.944)	1	Accept
Planted-positive control	18	0	18 (1.000)	17 (0.944)	1	Pass
Identity instrumentation	6	0	6 (1.000)	5 (0.833)	1	Pass

Table 2: Exact outcomes for the focused validation suite. Admission uses the frozen count-level rule for 18-block strict-subset families. The planted-positive and identity rows are measurement controls with separate pass criteria. Mean private scores summarize complete blocks; they are not population estimates.

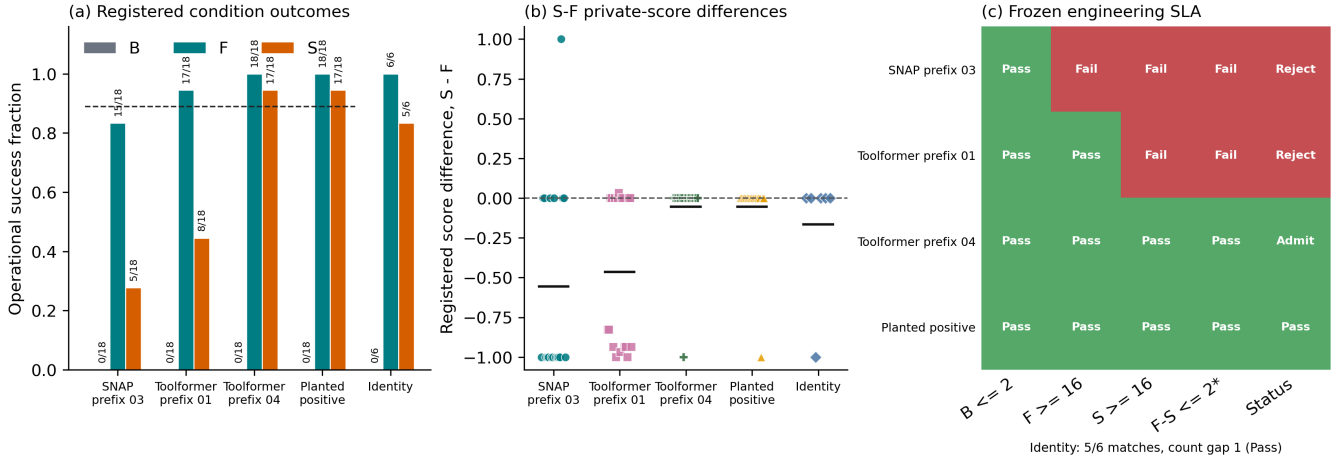


Figure 2: Focused finite-schedule evidence. (a) Exact operational-success fractions for B , F , and S ; the dashed line marks the 16/18 sufficiency threshold for strict-subset families. (b) Every registered block-level $S - F$ private-score difference, with black segments denoting finite-schedule means. (c) Field-level acceptance checks and resulting status. Toolformer prefix 04 was selected after the earlier focused comparison and evaluated on a separately frozen boundary; identity uses its separate paired-agreement rule.

RQ2: Decision and Measurement Sanity

The controls exercise three distinct properties. First, the planted-positive candidate removes only a registered redundant atom and passes at 0/18/17, showing that the gate does not reject every strict subset. Second, the real Toolformer 1/5 negative is rejected at 0/17/8, showing that the same task and scorer can expose a severe omission. Third, byte-identical F and S succeed in 6/6 and 5/6 fresh conversations. Five paired success indicators agree and the aggregate gap is one, so identity instrumentation passes, while the single mismatch demonstrates that identical instructions do not imply identical stochastic trajectories. These controls validate the exercised decision states; they are not a sensitivity/specificity estimate over a population of skills.

The baseline is also essential. A candidate cannot be credited when B already solves the task frequently. In the focused suite, all baseline counts are zero. In the broader stress test, one of the five operational successes occurs under B , illustrating why isolated successful rows cannot be interpreted as evidence for the candidate.

RQ3: Cross-Paper Scale and Failure Localization

The cross-paper execution produced all 1,296 terminal rows. At the primary task level, the frozen analysis yields 24 REJECT, 0 ACCEPT, and 0 INVALID decisions. Candidate yield is therefore 0/24. This result is not a clean estimate of candidate efficacy: the terminal distribution in Figure 3 is dominated by hard interface and integrity failures. SkillAudit’s useful output here is a diagnosis of the evidence boundary, not a claim that all 24 simplified specifications are semantically inadequate.

Only 5 individual rows meet the operational-success definition. Table 3 reports all five. Four are S rows and one is a B row; none is an F row. All have private score 1.0 and pass both hard-contract components. Three successes repeat the same LLMingua task and model across different blocks. Consequently, these rows do not form matched $B/F/S$ triples, and no defensible $F - S$ effect can be computed from them alone. Reporting a difference would confound artifact condition with task, block, registry, and model.

The dominant terminal category is 1,093 hard-contract failures, followed by 120 integrity/digest failures, 77 malformed or missing submissions, and 1 unavailable-model row. The interface audit reads 1,098 scoring documents

Task	Domain	Cond.	Model	Block/registry	Candidate tokens	q
AGENT-TF-02	Agent/tool use	S	DeepSeek V3.2	3/A	157	1.000
NLP-LLM-01	NLP	S	GPT-5.6 Sol	2/A	170	1.000
NLP-LLM-01	NLP	S	GPT-5.6 Sol	4/B	170	1.000
NLP-LLM-01	NLP	S	GPT-5.6 Sol	5/B	170	1.000
DATA-HDB-01	Data analysis	B	GPT-5.6 Sol	2/A	0	1.000

Table 3: All operationally successful rows in the 1,296-row cross-paper stress test. These five rows are terminal execution units, not complete task-level comparisons. In particular, their conditions are unbalanced (four S , one B , zero F), so they support protocol diagnosis but not a paired substitution effect.

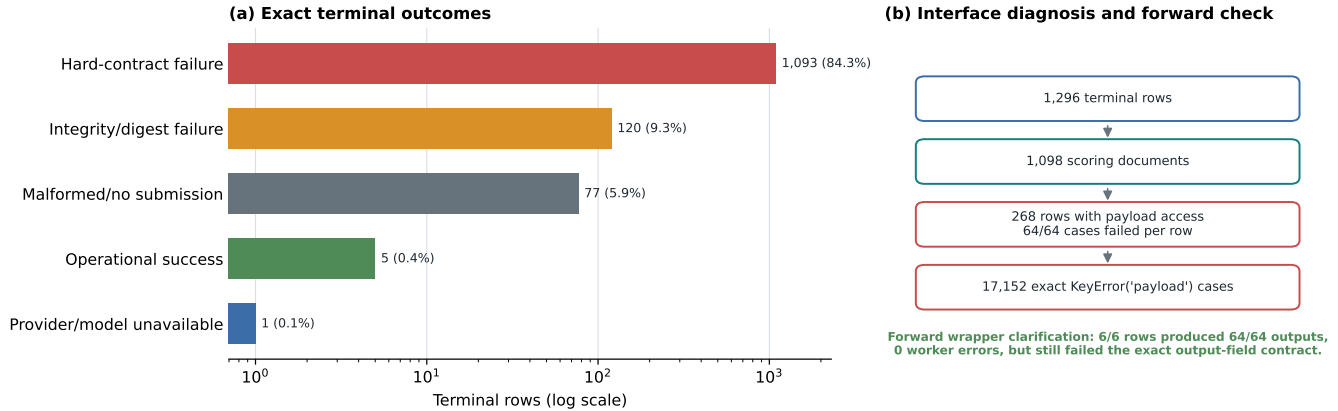


Figure 3: Failure decomposition for the cross-paper stress test. (a) Exact terminal outcomes over 1,296 registered rows; a log axis keeps the five successes and one unavailable-model row visible beside 1,093 hard-contract failures. (b) Interface diagnosis: 268 rows attempted to access a nonexistent outer payload wrapper and failed all 64 private cases, producing 17,152 exact `KeyError('payload')` cases. A forward wrapper clarification eliminated worker errors in six successor rows, but all six still failed the separately underspecified exact output-field contract.

and finds 268 affected execution rows with 17,152 exact `KeyError('payload')` cases. Every affected row fails 64/64 cases, and all 24 task scaffolds are ambiguous about whether `solve(case)` receives an outer wrapper or its inner payload.

We test this diagnosis with a forward-only interface clarification, without rewriting the 1,296 terminal rows. In six registered successor rows, the task scaffold explicitly states that `solve(case)` receives the inner payload. All 6 rows then produce 64/64 outputs with 0 worker errors, zero `KeyError('payload')`, and zero input mutations. They nevertheless receive private score zero because exact output field names remain underspecified. The models return semantically related fields such as `total_allocated` or list-valued allocations where the scorer requires `allocations`, `by_name`, and `total`. This successive localization is precisely why protocol validity and task success must be reported separately.

Robustness and Evidence Cost

Several design choices reduce avoidable confounding. Conditions are interleaved and order-balanced within the focused schedules; starter workspaces are isolated; private cases are terminal; output directories are write-once; and response identifiers, retries, patches, scores, and digests are retained.

The cross-paper suite duplicates private registries and varies reducer, retention level, model, and execution family. However, the interface defect prevents those dimensions from serving as a valid model-effect ablation. In particular, the four successes from GPT-5.6 Sol versus one from DeepSeek V3.2 cannot be interpreted as a model ranking.

The focused schedules contain 234 conversations, 1760 provider turns, 1760 recorded actions, and 1,776 transport attempts. They use 2,812,362 reported input tokens and 2,894,613 reported output tokens. The cross-paper run invokes a frozen retry policy and records 485 transport retries; only one terminal row remains unavailable. These totals document the cost of evidence generation. SkillAudit does not claim that validation itself is free or that shorter artifacts necessarily reduce end-to-end cost.

Discussion

Provenance and sufficiency answer different questions.

A source locator establishes where an instruction came from, and dependency closure establishes that retained procedural units include their declared prerequisites. Neither property shows that the resulting specification is sufficient for an agent. Conversely, a high private score without a source map cannot explain which paper content the artifact represents. SkillAudit joins these views without conflating them: the atom graph

explains the candidate boundary, while matched $B/F/S$ execution tests the local substitution claim.

The full specification is an active control, not an oracle. The SNAP result shows why F must be evaluated rather than assumed correct. When F is insufficient, a poor S score cannot be cleanly attributed to simplification. When F succeeds and S fails, as for Toolformer prefix 01, the candidate-specific diagnosis is stronger. When both succeed and B does not, as for prefix 04, acceptance is warranted only under the frozen task, model, tests, and thresholds. This three-way comparison is more informative than reporting candidate accuracy alone.

Invalidity is a scientific result. The cross-paper stress test does not provide the desired effectiveness evidence, but it exposes two concrete specification defects: ambiguity about the input wrapper and ambiguity about exact output fields. The forward clarification removes the first failure class without changing historical evidence and then reveals the second. Treating all of these rows as ordinary score zero would conceal the measurement problem. An auditable validator should preserve this distinction even when it lowers the apparent headline result.

Acceptance is deliberately local. An accepted candidate is not “the core” of a paper in the abstract. It is a candidate that substituted for F under (t, Ω) . A reusable release should therefore package S with its paper identity, atom boundary, task, model, scorer, private-registry identity, schedule, and field-level rationale. If any of those factors change, the previous record remains useful provenance but does not automatically authorize reuse.

Limitations and Broader Impact

The positive behavioral evidence is narrow. The focused suite covers two paper mechanisms, one primary provider model, one action scaffold, and finite 18-block schedules. Only one non-planted simplified candidate is accepted. Its selection used earlier evidence before a separate confirmatory boundary, so it is not a prospectively estimated discovery rate. The broad suite covers 12 papers and 24 tasks, but its 1,296 rows are dominated by interface failures. The five operationally successful rows are unbalanced across conditions and cannot support a paired effect. Full cross-paper efficacy therefore remains unestablished in this draft.

The atom boundaries, executable wording, dependencies, and completeness of F are evaluation-package judgments. Digests prove identity, not semantic correctness. Independent paper experts should review these structures, and future work should measure inter-annotator agreement over atomization and dependency edges. Atom restoration/removal experiments would also provide stronger causal evidence that a particular omitted source unit caused a candidate failure.

The acceptance thresholds are prespecified engineering parameters, not universal constants or calibrated population guarantees. The focused count rule tolerates fresh-session variability, but it does not establish trajectory equivalence. Provider-side sampling is not fully controllable: deterministic case seeds, ordering, and temperature settings do not

imply reproducible backend routing or sampling when the API exposes no enforceable random seed. Repetition across independently operated providers and additional hidden registries is needed before making broad robustness claims.

No human participants were studied. A user study is not required to test the internal validity of the finite substitution rule, but it is required for claims that SkillAudit helps researchers understand papers, reproduce methods faster, or avoid downstream scientific mistakes. Such a study should compare full specifications, unvalidated simplified candidates, and evidence-carrying accepted candidates on completion time, correction rate, and expert-rated fidelity.

SkillAudit may reduce overconfident reuse by making evidence boundaries visible. It may also create false assurance if an acceptance label is detached from its local task and scorer. Generated code and paper-derived tools retain normal supply-chain and execution risks; isolated workspaces, restricted tools, private scoring, and immutable evidence reduce but do not eliminate them. Releases should expose rejected and invalid cases as well as accepted ones to avoid selectively presenting only successful candidates.

Conclusion

We introduced SkillAudit, a traceable protocol for deciding whether a simplified paper-derived agent skill specification can replace its full counterpart under a frozen evaluation boundary. SkillAudit links specification units to exact source passages and dependencies, executes matched no-specification, full, and simplified conditions, validates evidence before scoring performance, and emits a local ACCEPT, REJECT, or INVALID record. In the focused suite, it distinguishes full-specification insufficiency from candidate-specific information loss, passes strict-subset and identity controls, and accepts one non-planted Toolformer candidate on separate held-out evidence. The 1,296-row cross-paper stress test further demonstrates the value of explicit invalidity diagnosis by localizing systematic input- and output-contract defects. These results support SkillAudit as an auditable validation layer, while leaving broad cross-paper effectiveness to a corrected, fully paired evaluation.

References

- Angelopoulos, A. N.; Bates, S.; Fisch, A.; Lei, L.; and Schuster, T. 2022. Conformal Risk Control. arXiv:2208.02814.
- Campello, R. J. G. B.; Moulavi, D.; Zimek, A.; and Sander, J. 2015. Hierarchical Density Estimates for Data Clustering, Visualization, and Outlier Detection. *ACM Transactions on Knowledge Discovery from Data*, 10(1): 5:1–5:51.
- Dwork, C.; Feldman, V.; Hardt, M.; Pitassi, T.; Reingold, O.; and Roth, A. 2015. The Reusable Holdout: Preserving Validity in Adaptive Data Analysis. *Science*, 349(6248).
- Edwards, N.; Lee, Y.; Mao, Y. A.; Qin, Y.; Schuster, S.; and Kim, N. 2026. RExBench: Can Coding Agents Autonomously Implement AI Research Extensions? In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 16380–16417.

- Gao, S.; Shi, Z.; Zhu, M.; Fang, B.; Xin, X.; Ren, P.; Chen, Z.; Ma, J.; and Ren, Z. 2024. Confucius: Iterative Tool Learning from Introspection Feedback by Easy-to-Difficult Curriculum. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 18030–18038.
- Gao, Y.; Li, Z.; Yuan, Y.; Ji, Z.; Ma, P.; and Wang, S. 2026. SkillReducer: Optimizing LLM Agent Skills for Token Efficiency. arXiv:2603.29919.
- Geifman, Y.; and El-Yaniv, R. 2019. SelectiveNet: A Deep Neural Network with an Integrated Reject Option. arXiv:1901.09192.
- Groce, A.; Alipour, M. A.; Zhang, C.; Chen, Y.; and Regehr, J. 2016. Cause Reduction: Delta Debugging, Even without Bugs. *Software Testing, Verification and Reliability*, 26(2).
- Guo, Z.; Xu, B.; Zhu, C.; Hong, W.; Wang, X.; and Mao, Z. 2026. MCP-AgentBench: Evaluating Real-World Language Agent Performance with MCP-Mediated Tools. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 30888–30896.
- Hao, Y.; Cao, P.; Jin, Z.; Liao, H.; Chen, Y.; Liu, K.; and Zhao, J. 2025. CITI: Enhancing Tool Utilizing Ability in Large Language Models Without Sacrificing General Performance. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 23996–24004.
- Hu, C.; Zhang, L.; Lim, Y.; Wadhwani, A.; Peters, A.; and Kang, D. 2025. REPRO-Bench: Can Agentic AI Systems Assess the Reproducibility of Social Science Research? In *Findings of the Association for Computational Linguistics: ACL 2025*, 23616–23626.
- Jia, J.; and Li, Q. 2026. AutoTool: Efficient Tool Selection for Large Language Model Agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 31265–31273.
- Jiang, H.; Wu, Q.; Lin, C.-Y.; Yang, Y.; and Qiu, L. 2023. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 13358–13376.
- Jiang, H.; Wu, Q.; Luo, X.; Li, D.; Lin, C.-Y.; Yang, Y.; and Qiu, L. 2024. LongLLMingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.
- Larionov, D.; and Eger, S. 2025. PromptOptMe: Error-Aware Prompt Compression for LLM-based MT Evaluation Metrics. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 11807–11820.
- Liskavets, B.; Ushakov, M.; Roy, S.; Klibanov, M.; Etemad, A.; and Luke, S. K. 2025. Prompt Compression with Context-Aware Sentence Encoding for Fast and Improved LLM Inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 24595–24604.
- Liu, D.; Li, Z.; Du, H.; Wu, X.; Gui, S.; Kuang, Y.; and Sun, L. 2026. Graph-of-Skills: Dependency-Aware Structural Retrieval for Massive Agent Skills. arXiv:2604.05333.
- Luo, Y.; Yu, Z.; Wang, X.; Zhu, Y.; Zhang, N.; Wei, L.; Du, L.; Zheng, D.; and Chen, H. 2026. What Makes AI Research Replicable? Executable Knowledge Graphs as Scientific Knowledge Representations. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 841–861.
- Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. arXiv:2401.13178.
- Ma, Z.; Huang, Z.; Liu, J.; Wang, M.; Zhao, H.; and Li, X. 2025. Automated Creation of Reusable and Diverse Toolsets for Enhancing LLM Reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 24821–24830.
- Meng, X.; Wang, S.; and Fang, Y. 2026. SkillRAE: Agent Skill-Based Context Compilation for Retrieval-Augmented Execution. arXiv:2605.10114.
- Miao, J.; Davis, J. R.; Zhang, Y.; Pritchard, J. K.; and Zou, J. 2025. Paper2Agent: Reimagining Research Papers as Interactive and Reliable AI Agents. arXiv:2509.06917.
- Nakkiran, P.; and Błasiok, J. 2018. The Generic Hold-out: Preventing False-Discoveries in Adaptive Data Science. arXiv:1809.05596.
- Pan, Q.; Yang, Y.; Li, J.; Zhou, J.; Chen, K.; Li, X.; Chen, Q.; and He, L. 2026. Anything2Skill: Compiling External Knowledge into Reusable Skills for Agents. arXiv:2606.09316.
- Pan, Z.; Wu, Q.; Jiang, H.; Xia, M.; Luo, X.; Zhang, J.; Lin, Q.; Ruhle, V.; Yang, Y.; Lin, C.-Y.; Zhao, H. V.; Qiu, L.; and Zhang, D. 2024. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- Regehr, J.; Chen, Y.; Cuoq, P.; Eide, E.; Ellison, C.; and Yang, X. 2012. Test-Case Reduction for C Compiler Bugs. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 335–346.
- Schick, T.; Dwivedi-Yu, J.; Dessi, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36*.
- Shen, S.; Cheng, W.; Ma, M.; Turcan, A.; Zhang, M. J.; and Ma, J. 2026. SKILLFOUNDRY: Building Self-Evolving Agent Skill Libraries from Heterogeneous Scientific Resources. arXiv:2604.03964.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36*.
- Starace, G.; Jaffe, O.; Sherburn, D.; Aung, J.; Chan, J. S.; Maksin, L.; Dias, R.; Mays, E.; Kinsella, B.; Thompson, W.;

- Heidecke, J.; Glaese, A.; and Patwardhan, T. 2025. Paper-Bench: Evaluating AI’s Ability to Replicate AI Research. In *Proceedings of the 42nd International Conference on Machine Learning*.
- Sun, C.; Li, Y.; Zhang, Q.; Gu, T.; and Su, Z. 2018. Perses: Syntax-Guided Program Reduction. In *Proceedings of the 40th International Conference on Software Engineering*, 361–371.
- Tang, J.; Xu, J.; Lu, T.; Zhang, Z.; Zhao, Y.; Hai, L.; and Zheng, H.-T. 2025. Perception Compressor: A Training-Free Prompt Compression Framework in Long Context Scenarios. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 4093–4108.
- Traag, V. A.; Waltman, L.; and van Eck, N. J. 2019. From Louvain to Leiden: Guaranteeing Well-Connected Communities. *Scientific Reports*, 9: 5233.
- Wang, Q.; Hu, Y.; Lu, M.; Wu, J.; Liu, Y.; and Tang, Y. 2026. Beyond Accuracy: A Cognitive Load Framework for Mapping the Capability Boundaries of Tool-Use Agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 33611–33619.
- Xu, X.; and Wu, W. 2026. Compression, Structure, and Executor Capability: A Controlled Real-Cost Decomposition of Language-Model Agent Skill Optimisation. arXiv:2607.03048.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.
- Yuan, B.; Su, Y.; Song, R.; Yang, S.; and Qin, J. 2026. Claw-Trace: Cost-Aware Tracing for LLM Agent Skill Distillation. arXiv:2604.23853.
- Zeller, A.; and Hildebrandt, R. 2002. Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering*, 28(2): 183–200.
- Zhang, K.; Zemke, N. R.; Armand, E. J.; and Ren, B. 2024. A Fast, Scalable and Versatile Tool for Analysis of Single-Cell Omics Data. *Nature Methods*, 21: 217–227.
- Zhao, X.; Sang, Z.; Li, Y.; Shi, Q.; Zhao, W.; Wang, S.; Zhang, D.; Han, X.; Liu, Z.; and Sun, M. 2026. AutoReproduce: Automatic AI Experiment Reproduction with Paper Lineage. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 21920–21942.